

Health-Checked Migration

Proof of Concept

Pre-Validation of Preferred Servers Before QUIC Connection Migration

PoC Documentation • July 2026

This document describes the Health-Checked Migration proof of concept, which adds pre-validation of the preferred server before advertising `preferred_address` in the QUIC handshake. Without health checking, a primary server unconditionally advertises a preferred address; if the preferred server is down, the client experiences a `PATH_CHALLENGE` timeout (100-600ms). Our PoC implements two health check modes -- **TCP Probe** and **Redis Heartbeat** -- that allow the primary server to detect preferred server failures and gracefully fall back to serving clients directly. The result is zero user-visible errors during preferred server outages, with negligible added latency (<0.2ms per connection).

Contents

1. Overview
 - 1.1 The Problem
 - 1.2 The Solution
2. Architecture
 - 2.1 Testbed
 - 2.2 Health Check Modes
3. Implementation Details
 - 3.1 Binaries and Configuration
 - 3.2 Background Health Thread
 - 3.3 Conditional preferred_address
 - 3.4 HTML Status Page
4. How to Run
5. Test Scenarios
6. Expected Results
7. Comparison Table

1. Overview

1.1 The Problem

In our current QUIC server-side migration implementation, the primary server **unconditionally** advertises a `preferred_address` transport parameter during the TLS handshake. This tells the client to migrate its connection to the preferred server's address. However, if the preferred server is down or unreachable at the time of migration, the client sends a `PATH_CHALLENGE` to an unresponsive host and experiences a timeout.

Problem: The client has no way to know the preferred server is down until `PATH_CHALLENGE` times out (100-600ms depending on implementation). This causes user-visible latency spikes and potential connection failures.

1.2 The Solution

The solution is to add a **health check** mechanism: the primary server verifies that the preferred server is alive and ready before advertising `preferred_address` in the handshake. If the preferred server is unhealthy, the primary simply serves the client directly without migration -- the client never knows migration was an option.

Result: Zero failed migrations. If the preferred server is down, clients are served directly by the primary with no timeout or error.

2. Architecture

2.1 Testbed

The PoC uses the same four-machine LAN testbed as our existing migration implementation:

Role	Machine	IP Address	Function
Client	optiplex7010	141.217.168.127	Firefox browser (HTTP/3)
Primary Server	opti7040	141.217.168.152	Completes TLS handshake, runs health checks
Preferred Server	homeserver2	141.217.168.143	Imports state, handles <code>PATH_CHALLENGE</code>
Redis Server	Proxmox VM	141.217.168.200	Heartbeat key storage (Redis mode)

2.2 Health Check Modes

The PoC supports two complementary health check strategies, each with different trade-offs in detection latency, infrastructure requirements, and failure modes:

TCP Probe

The primary server performs a TCP connect to the preferred server's state-transfer port (9999) with a **200ms timeout**. This is a per-connection probe: before each new QUIC handshake, the primary attempts a TCP connection to the preferred server. If the connection succeeds, the preferred server is considered healthy and `preferred_address` is advertised. If the connection fails or times out, the primary serves the client directly.

- Per-connection granularity -- every handshake is individually validated
- No external infrastructure required (no Redis dependency)
- Detection latency: bounded by the 200ms TCP timeout
- Trade-off: one extra TCP round-trip per connection (~0.1ms on LAN)

Redis Heartbeat

The preferred server publishes a `preferred:health` key to Redis every **1 second** with a **3-second TTL**. The primary server checks this key before each handshake. If the key exists, the preferred server has been alive within the last 3 seconds. If the key has expired (TTL elapsed), the preferred server is considered down.

- Decoupled: preferred server does not need to be directly reachable from primary
- Requires Redis infrastructure (already available on our testbed at .200)
- Detection latency: up to 3s (heartbeat TTL) after preferred server failure
- Trade-off: one Redis GET per connection (~0.1ms on LAN)

3. Implementation Details

3.1 Binaries and Configuration

The PoC introduces two new binaries in the `migration-test/` crate:

Binary	Description
health-check-primary	Modified primary server with health check logic. Conditionally advertises <code>preferred_address</code> based on health status.
health-check-preferred	Preferred server with Redis heartbeat publishing. Sends periodic health updates to Redis.

Environment Variables

Variable	Values	Description
HEALTH_CHECK	tcp redis none	Selects the health check mode. Default: none (unconditional migration).
REDIS_URL	redis://141.217.168.200:6379	Redis connection URL for heartbeat mode. Only used when HEALTH_CHECK=redis.

3.2 Background Health Thread

The primary server spawns a background thread that re-checks the preferred server's health every **5 seconds**. This thread maintains a shared atomic boolean (`is_healthy`) that the main connection-handling code reads before each handshake. Status changes are logged:

```
[health] preferred server status changed: HEALTHY -> UNHEALTHY
[health] preferred server status changed: UNHEALTHY -> HEALTHY
```

The background thread provides resilience against transient failures and enables automatic recovery when the preferred server comes back online.

3.3 Conditional `preferred_address`

The key modification is in the connection setup path. Before constructing `ConnectionParameters`, the primary checks the health status:

```
if is_healthy.load(Ordering::Relaxed) {
    // Set preferred_address in ConnectionParameters
    // Export and transfer migration state after handshake
} else {
    // Serve client directly, no preferred_address
    // Log: "preferred server unhealthy, serving directly"
}
```

When healthy, the server behaves identically to the existing migration flow. When unhealthy, the server acts as a normal HTTP/3 server with no migration.

3.4 HTML Status Page

The HTTP/3 response page dynamically reflects the migration status. When the preferred server is healthy, the page shows a green "HEALTHY" badge and indicates that migration is active. When unhealthy, the page shows a red "UNHEALTHY" badge and states that the client is being served directly by the primary.

4. How to Run

The PoC is launched from the same machines as the existing migration demo. The preferred server is started first, followed by the primary.

Step 1: Start the Preferred Server (homeserver2)

```
health-check-preferred 141.217.168.143:4433 9999
```

This starts the preferred server listening on port 4433 (QUIC) and port 9999 (state transfer / TCP probe endpoint). In Redis mode, it also begins publishing heartbeats to Redis every 1 second.

Step 2: Start the Primary Server (opti7040)

```
HEALTH_CHECK=tcp health-check-primary 0.0.0.0:4433 \  
141.217.168.143:4433 141.217.168.143:9999
```

Arguments: (1) local bind address, (2) preferred server QUIC address, (3) preferred server state-transfer address. The HEALTH_CHECK variable selects the probe mode.

Step 3: Open Firefox

Navigate to <https://141.217.168.152:4433/>. The bootstrap server (Alt-Svc) must be running on opti7040 for the first HTTP/3 upgrade, same as the standard demo. After the first load, refresh to use HTTP/3 directly.

Tip: To switch between TCP and Redis modes, simply change the HEALTH_CHECK environment variable and restart the primary server. No changes needed on the preferred server side.

5. Test Scenarios

Baseline: Both Servers Up

1. Start both servers normally.
2. Load the page in Firefox -- migration succeeds as usual.
3. Expected: 100% successful migration, identical to existing demo.
4. Purpose: Confirms health check adds no regression.

Preferred Down: Kill Preferred Server

1. Start both servers, verify migration works.
2. Kill the preferred server (preferred-down on homeserver2).
3. Wait 5 seconds for the background health thread to detect the failure.
4. Load the page in Firefox again.
5. Expected: Primary detects unhealthy preferred, serves directly. Zero timeout, zero errors. Page loads normally.

Flapping: Start/Stop Preferred Every 5s

1. Start both servers.

2. In a loop, alternate: kill preferred, wait 5s, start preferred, wait 5s.
3. Load the page repeatedly during the flapping cycle.
4. Without health check: random PATH_CHALLENGE timeouts on ~50% of loads.
5. With health check: graceful fallback, zero timeouts.

Recovery: Preferred Comes Back Up

1. Start both servers, then kill the preferred server.
2. Verify primary serves directly (no migration).
3. Restart the preferred server.
4. Wait up to 5 seconds for the background health thread to detect recovery.
5. Load the page -- migration should resume.
6. Expected: Automatic recovery with no manual intervention.

6. Expected Results

The health check mechanism is expected to eliminate all user-visible failures during preferred server outages:

Without health check: When the preferred server is down, clients experience PATH_CHALLENGE timeouts of 100-600ms. In the worst case, the connection may fail entirely if the client's retry budget is exhausted.

With health check: Zero failed migrations. The primary detects the preferred server is down and serves clients directly. The transition is invisible to the client -- no timeout, no retry, no error.

Key Metrics

- **Failed migrations:** 0% (vs. 100% without health check when preferred is down)
- **Added latency (healthy path):** ~0.1ms per connection (TCP connect or Redis GET)
- **Detection latency (TCP):** <200ms (bounded by TCP timeout, per-connection)
- **Detection latency (Redis):** <3s (bounded by heartbeat TTL)
- **Recovery time:** <5s (background thread polling interval)
- **User-visible errors:** Zero during preferred server failures

7. Comparison Table

The following table compares behavior across all scenarios with and without health checking enabled:

Scenario	Without Health Check	With Health Check (TCP)	With Health Check (Redis)
Preferred healthy	Migration OK	Migration OK	Migration OK
Preferred down	Client timeout (100-600ms)	Serves directly	Serves directly
Preferred flapping	Random timeouts	Graceful fallback	Graceful fallback
Detection latency	N/A (no detection)	<200ms (per-conn probe)	<3s (heartbeat TTL)
Added latency	0	~0.1ms (TCP connect)	~0.1ms (Redis GET)

Trade-off Summary

Aspect	TCP Probe	Redis Heartbeat
Infrastructure	None (direct TCP)	Requires Redis server
Granularity	Per-connection	Periodic (1s heartbeat)
Detection speed	Immediate (<200ms)	Delayed (up to 3s TTL)
Network topology	Primary must reach preferred directly	Both must reach Redis; no direct link needed
Failure mode	Probe fails = no migration (safe)	Redis down = key absent = no migration (safe)
Best for	Low-latency, simple setups	Multi-datacenter, indirect routing

Both modes fail safe: if the health check itself fails (e.g., Redis is unreachable), the primary defaults to serving directly without migration. This ensures that health check infrastructure failures never cause client-visible errors.